Project1：祖玛游戏实验报告

项目	内容
姓名	甘锦珊
学号	24325083
课程	数据结构
日期	2026 年 11 月 7 日

一、项目概述

祖玛游戏：若干不同颜色的彩球沿固定轨道依次排列，玩家不断发射新球并插入轨道上的彩球序列中；当轨道上出现连续三个或三个以上相同颜色的彩球时，这些彩球被自动消除，消除后原本分开的两段彩球重新连接，若再次形成三连及以上则继续消除，这一现象称为连锁消除。

本项目在简化规则下（轨道上的球静止不动、玩家每次从待发射队列取球插入指定位置）实现祖玛的核心玩法，并用**栈、队列和链表**三种数据结构分别支撑游戏的不同环节：

- 彩球**轨道** → 双向链表；
- 待发射**队列** → 循环队列；
- **撤销**操作 → 栈。

程序实现了题目要求的全部功能——彩球轨道初始化、彩球发射、指定位置插入、自动消除、连锁消除、撤销操作、游戏胜负判断，并额外设计了两个小道具（炸弹、改色）以增强可玩性、进一步体现数据结构的运用。程序提供交互模式（真人游玩）与批处理模式（自动测试）两种运行方式，前者面向玩家，后者面向测试脚本。

二、设计思路

2.1 总体架构

整体思路是把游戏状态拆成三个独立的数据结构，再用一个 Game 层组合起来：

层	组件	职责
数据结构层	`Track`	双向链表 + 首尾哨兵，表示轨道
	`BallQueue`	环形缓冲区（循环队列），表示待发射球
	`UndoStack`	栈，保存历史快照，支持撤销
组合层	`Game`	持有三者，实现 fire / bomb / paint / undo 与胜负判定
输入层	`main.cpp`	`--script` 批处理模式；无参数进入交互模式
	`interactive.cpp`	命令行界面：预设关卡、局面显示、提示与播报

2.2 数据结构选型与理由

轨道 —— 手写双向链表 + 首尾哨兵。 消除算法需要以某颗球为起点向**左**和向**右**两个方向同时扫描同色段，删掉一段之后又要把它左右两边的球重新接上。单链表无法向左走；而数组在中间删除需要整体搬移，一次消除的代价是 `O(n)`，连锁消除还会反复发生。双向链表在已经持有节点指针时，删除与两端访问都是 `O(1)`，天然契合这个场景。头尾各放一个哨兵节点，使链表永远非空，插入/删除不必对空链表、头部、尾部做特判；哨兵的颜色固定为 `'\0'`（不是合法彩球色），`Game` 层据此即可判断左右是否还有球，避免把哨兵误当成彩球参与比较。

待发射队列 —— 手写循环队列（环形缓冲区）。 游戏只用到队首出一颗、队尾进一颗两种操作。普通数组队列出队后需要整体前移（`O(n)`）；循环队列让 `front/pop/push` 都是 `O(1)`，并且复用同一块缓冲区，反复发射不会产生新的分配。用额外一个空槽区分空/满（底层分配 `cap+1` 个槽），不额外占一个标记位。

撤销 —— 栈（快照法）。 每次状态改变前把整份游戏状态压栈，UNDO 时弹出栈顶恢复，栈的后进先出语义天然对应撤销上一步。底层用 `std::vector` 但只暴露栈接口，不使用 `std::stack`。

三者分工明确、互不耦合：`Track` 只管链表、`BallQueue` 只管队列、`UndoStack` 只管历史，`Game` 只负责编排与胜负，任何一层都可以单独编译、单独测试。

2.3 开发过程

本项目开发思路为：把任务拆分为 6 个阶段（阶段 0 至阶段 5），每个阶段以一段明确的 Prompt 提供给智能体，智能体实现并汇报后，再进入下一阶段。下面逐阶段概括每段 Prompt 的**思路**、智能体**回应完成的工作**，并附上对应截图作为过程证据。

阶段 0 · 总纲

Prompt 思路：先定好交付结构与通用约定，避免后续返工。核心是三件事——(1) 交付目录布局（`src/` 分文件、`inputs/`+`outputs/` 样例、`test` 测试程序、`Makefile`）；(2) 数据结构选型（轨道=双向链表、发射队列=循环队列、撤销=栈）；(3) `--script` 批处理模式的**确定性 I/O 格式**，让测试程序能逐行精确比对。同时约定跨平台细节（`EXEEXT`、`resolve_exe`）。

```
› 我们之前已经完成了MyLab下的Linear/Baseline任务，下面我们将继续完成Linear/Project任务，要求更改仍然仅仅在MyLab目录下进行。下面我将给你Project的计划，按它执行：Project/README.md
  作为背景和样例参考。有疑问可以随时询问我，不要任意猜测。明白之后反馈给我，我们开始第一阶段工作。

  Searched for 2 patterns, read 1 file, listed 1 directory (ctrl+o to expand)
```

智能体工作：确认已理解交付结构与 I/O 约定，等待确认后开始实现（本阶段不产代码）。

```
● 我已经读完了相关材料。先反馈我的理解，再提两个问题。

  我确认的背景

  任务目标：在 MyLab/Linear/Project/ 下完成 src/Linear/Project/README.md 描述的祖玛（Zuma）综合项目。

  题目核心要求（来自 src/Linear/Project/README.md）：

  - 彩球轨道 + 待发射彩球队列 + 游戏状态，需要显式设计数据结构
  - 必需功能：轨道初始化、发射彩球、指定位置插入、自动消除（≥3 同色）、连锁消除、撤销、胜负判断
  - 额外小道具功能（计入评分）
  - 交互界面（CLI 必须，GUI 加分不强制）
  - 提交物：inputs/caseMM.txt + outputs/caseMM.txt、名为 test 的自动比对程序、.cpp 源码、一条命令完成编译的 Makefile
  - 评分：功能 30% / 数据结构 30% / 交互 20% / 代码质量 20%

  我会延续的既有约定（从 MyLab/Linear/Baseline/ 两个实验沿用）：

  - 所有改动只在 MyLab/ 下，不碰仓库根目录和 src/
  - C++17，-Wall -Wextra -Wpedantic，零 warning
  - Makefile：$(EXEEXT) / $(PYTHON) 跨平台，目标为 all / test / clean
  - test 为 Python 脚本，逐行比对同时忽略行尾空白、末尾空行和 PERF 行
  - 元素存储不依赖 STL 容器（这个项目里轨道/队列/撤销栈都自己实现，正好对应考点）
  - 附 README.md + ai_record.md 记录设计思路与逐轮协作记录

  当前目录状态：MyLab/Linear/Project/ 目前只有一份 README.md，其余为空。另外我注意到 MyLab/Linear/Baseline/ 是未跟踪的新目录、旧的 MyLab/Linear/list、StackAndQueue
  显示为已删除——这是你上次整理的结果，我不会去动它。

  需要你确认的两点

  1. Project 的计划没有附上。你消息里说"下面我将给你Project的计划"，但正文在"开始第一阶段工作"处就结束了，我这边没有收到计划内容。请把计划贴出来。
  2. 项目 README 的副本：MyLab/Linear/Project/README.md 是 src/Linear/Project/README.md 的拷贝吗？我计划保留它不动（作为背景参考），另外新建项目自己的 README.md 和
     ai_record.md。如果希望直接在这份文件上扩展，请告诉我。

  计划收到后我就开始第一阶段。
```

阶段 1 · 三个核心数据结构

Prompt 思路：手写三个数据结构并保证各自可独立编译、可自检——`Track`（双向链表 + 首尾哨兵，手动 `new/delete`）、`BallQueue`（环形缓冲区，用额外一个空槽判空/满）、`UndoStack`（栈接口封装的 `std::vector`）,先不做游戏逻辑。

```
› 在 MyLab/Linear/Project/src/ 完成三个手写数据结构，各自能独立编译。先不写游戏逻辑。
  track.h/.cpp、ball_queue.h/.cpp、undo_stack.h/.cpp。

  1. Track —— 双向链表（轨道）
    namespace zuma { class Track { ... }; }
    私有：struct Node { char color; Node* prev; Node* next; }; 头尾哨兵节点 head_/tail_、
        size_t size_。
    接口：
      Node* insertAt(size_t pos, char color);  // 0 ≤ pos ≤ size，返回新节点
      void  erase(Node* p);                    // 从链表摘下并 delete
      Node* nodeAt(size_t pos);                // 0 ≤ pos < size，返回节点（供消除逻辑用）
      size_t size() const;  bool empty() const;
      std::vector<char> toVector() const;      // 头→尾，供打印/快照/比对
      void  clear();
    Node 的 prev/next 和 color 允许 Game 层读取（消除逻辑要用），可把 Node 设为
    public 嵌套结构或提供 getPrev/getNext/colorOf 访问器，二选一，别把哨兵暴露出去。
```

智能体工作：完成 `track.h/.cpp`、`ball_queue.h/.cpp`、`undo_stack.h/.cpp` 三个类；写了临时自检程序，覆盖 pos=0/中间/末尾插入删除、clear 后复用、环形回绕、空

队异常、LIFO 顺序等边界，在普通构建与 ASan/UBSan 下全部通过，验证后删除临时自检。确立了哨兵 `color='\0'` 这一 Track 与 Game 之间的约定。期间修复了Track 构造时第二个哨兵分配失败会泄漏第一个的异常安全细节。

- 阶段 1 完成。

交付文件（MyLab/Linear/Project/src/）

目前只有这 6 个文件，临时测试已删除，src/ 里没有多余产物。

track.h / track.cpp —— zuma::Track

接口	说明
Node* insertAt(size_t pos, char color)	0 ≤ pos ≤ size；pos == size 走尾插 O(1)；越界抛 std::out_of_range 且不改链表
void erase(Node* p)	摘链并 delete；nullptr/哨兵抛 std::invalid_argument
Node* nodeAt(size_t pos)	0 ≤ pos < size；从近端遍历，最坏 O(n/2)；越界抛 std::out_of_range
size() / empty() / clear() / toVector()	clear 只回收彩球、保留哨兵，可复用

Node* nodeAt(size_t pos)	0 ≤ pos < size，从较近一端遍历，最坏 O(n/2)
size_t size() const / bool empty() const	O(1)
std::vector<char> toVector() const	头→尾
void clear()	只释放彩球，保留哨兵，清空后可复用

内部：head_/tail_ 哨兵 + size_，节点全部手动 new/delete，全程无 std::list。构造函数对第二个哨兵分配失败做了兜底释放。

内部：head_/tail_ 哨兵 + size_，节点全部手动 new/delete，全程无 std::list。构造函数对第二个哨兵分配失败做了兜底释放。

ball_queue.h / ball_queue.cpp — zuma::BallQueue

接口	说明
explicit BallQueue(size_t capacity = 0)	底层分配 capacity + 1 槽；容量 0 时恒空
bool push(char c)	满返回 false，队列不变
char front() const	空抛 std::out_of_range
char pop()	出队并返回；空抛 std::out_of_range
void setFront(char c)	PAINT 用；空抛 std::out_of_range
size()/empty()	O(1)
toVector()	队首→队尾
clear()	只重置下标，保留缓冲区与容量
拷贝构造 / 赋值	按 FIFO 顺序重建，结果布局紧凑

内部：char* buf_（new char[]）、front_、back_、size_、cap_；front_ == back_ 为空，nextIndex(back_) == front_ 为满，无额外标记位。全程无 std::queue。

undo_stack.h / undo_stack.cpp — zuma::UndoStack + zuma::GameState

接口	说明
struct GameState { std::vector<char> track; std::vector<char> queue; }	阶段 1 最小占位，阶段 2 补字段
void push(const GameState&)	压栈
const GameState& top() const	空抛 std::out_of_range（按你列的接口清单补齐的读取入口）
bool pop(GameState& out)	空栈返回 false 且不改 out；非空时移动出栈顶再弹出
bool empty() const / size_t size() const / void clear()	—

内部 std::vector<GameState> states_ 只做栈尾操作，头文件注释写明「这里用栈管理撤销历史」，不暴露下标访问，无 std::stack。

阶段 2 • 游戏核心逻辑与批处理驱动

Prompt 思路：实现 `Game` 的核心玩法——初始化、发射（队首出队 + 指定位置插入）、自动消除、连锁消除，并让 `main.cpp --script` 跑通基本样例。重点是 `eliminateFrom(Node*)` 的找极大同色段 → ≥3 删除 → 左右同色则连锁算法。

```
❯ erase 返回 void，删除前先存 left->prev 和 right->next 就行，不用改接口。现在，在 MyLab/Linear/Project/src/ 完成 Game 核心玩法，并让 main.cpp 的 --script
  模式能跑通基本样例。完成game.h/.cpp、main.cpp（先只做 --script 批处理，交互模式之后 再做）。

  Game 类（namespace zuma） 持有：Track track_; BallQueue queue_; UndoStack undo_; 以及当前胜负状态。
    关键方法：
      void  init(const std::vector<char>& track, const std::vector<char>& queue);
      bool  fire(size_t pos, std::string* out_track, bool* won, bool* lost);
      bool  undo(std::string* out_track);           // 恢复后轨道
      bool  bomb(size_t pos, std::string* out_track, bool* won);
      bool  paint(char c, std::string* out_queue);
      std::string trackStr() const;  std::string queueStr() const;  // 供 SHOW/打印
    规则（与阶段 0 的 I/O 约定严格一致）：
      - fire：队首出队 → track_.insertAt(pos, color) → eliminate(newNode) → 判胜负。
      - bomb：erase(nodeAt(pos)) → 若删除导致左右相邻同色，做一轮合并消除（连锁）。
      - undo：undo_.pop 快照，恢复 track_ 和 queue_。
      - 每次 fire/bomb/paint 成功前，先把当前 GameState 压入 undo_（paint 也要压，
        这样 undo 也能撤销 paint）。
```

智能体工作：完成 `game.h/.cpp`、`main.cpp` 及 `case01/case02` 样例。`eliminateFrom` 采用找段 → 删段 → 换探针的循环，删段前先记下 `left_ball`/`right_ball`，只有两者都存在且同色才把探针移过去继续连锁。`GameState` 补齐了 `won`/`lost` 标志（否则FIRE 判负后 UNDO会残留胜负状态）。用独立 Python 参考实现做 3000 组随机差分测试，0 处不一致；ASan/UBSan 下复跑同样通过。

```
比对结果

./build/zuma --script < inputs/case01.txt   # EMPTY / WIN          → 与 outputs/case01.txt 一致
./build/zuma --script < inputs/case02.txt   # 5 行，逐行一致       → PASS

额外验证：写了一份独立 Python 参考实现做随机差分测试，3000 组随机脚本（含越界 pos、负数下标、非法颜色、连续 UNDO、BOMB/PAINT/FIRE 混合）0 处不一致；同一套用例在 ASan/UBSan +
泄漏检查下重跑，同样 0 不一致、无告警。临时脚本与临时二进制已删除。

eliminateFrom 的连锁处理

实现是一个「找段 → 删段 → 换探针」的循环，探针永远指向当前真正受影响的节点：

1. 从 probe 向左右扩展，得到包含它的极大同色段 [first, last]，段长 ≥3 才继续。
2. 删除整段之前先记下 left_ball = first->prev、right_ball = last->next（'\0' 色即哨兵，记作 nullptr）。删除循环里先取 next 再 erase，因为 erase 之后节点已失效。
3. 删除后判断：左右都存在且同色 → probe = left_ball，进入下一轮重新找极大段——这就是连锁；否则 probe = nullptr，循环结束。

对三种关键情形都成立：

- 左/右不足：任一侧为 nullptr 时合并段不可能达到 3，直接结束。
- 合并后 <3：下一轮找到的极大段长度就是合并后的长度，length < 3 分支 break，不会误删。
- 连续多级：R R B B B R R → 删 B B B → 探针落在左 R，极大段 R R R R ≥3 → 再删 → 探针为空（两端都是哨兵）→ 结束，轨道为空 → WIN。case01 正是这条路径。

bomb 只在「删除点原本的左右邻居同色」时才调用 eliminateFrom(left)，其余位置不可能新产生 ≥3 段，与题目规则一致。

阶段 2 顺带的两处补充（都不改你定的接口）

- GameState 加了 won / lost 两个字段。否则「FIRE 造成 LOSE 后 UNDO」会让胜负标志残留——快照必须能完整描述状态。
- Track::Node 的哨兵 color 固定为 '\0'，已在 track.h 注释里写成 Track 与 Game 之间的约定，Game 用 isBall() 判断是否走到端点，不接触哨兵指针本身。

需要你确认的规则细节

1. LOSE 之后是否拒绝继续操作？ 我现在没有拦截：判负后 FIRE 会继续判 ERROR（队列空），但 UNDO / BOMB 仍可执行。批处理脚本里若出现这种情况，是否应一律输出 ERROR？
2. BOMB 是否也压撤销栈？ 按你给的范围「fire/bomb/paint 成功前都压」已实现，UNDO 能撤销炸弹。确认一下这符合预期（题面只说「撤销操作」，没限定操作类型）。

下一阶段如果是交互模式 + Makefile + test，我按计划继续。
```

阶段 3 • 撤销 / 胜负 / 小道具

Prompt 思路：补全撤销、胜负判定与小道具，并补样例。要求 UNDO 能回退 fire/bomb/paint 三种操作、空栈报错；胜负统一在 fire/bomb 后判定；道具 `BOMB`（炸一颗球）与 `PAINT`（改队首颜色）都可撤销。

```
❯ 这两个问题都可以确认。现在，在 MyLab/Linear/Project/src/ 补全撤销、胜负判定和小道具，并补样例。

  撤销：阶段 2 已实现快照栈，本阶段重点核验：
    - UNDO 能回退 fire/bomb/paint 三种操作，恢复轨道与队列。
    - UNDO 栈空时输出 ERROR 且不改状态。
    - 撤销后继续操作应正常（比如 undo 后再次 fire 用的是恢复后的队列）。

  胜负判定：统一在每次 fire/bomb 后判定：
    - 轨道清空 → WIN（游戏结束）。
    - FIRE 后队列空且轨道非空 → LOSE（没球可打了）。
    输出顺序：先打印轨道行，再打印 WIN 或 LOSE（若触发）。

  小道具（至少实现这两个，可再加）：
    - BOMB pos：删除轨道 pos 位置一颗球，再触发消除与连锁消除。
      （用法/触发方式在交互模式里说明；批处理模式直接用 BOMB pos 命令。）
    - PAINT c  ：把待发射队列队首球颜色改成 c（不消耗球、不改轨道）。
    这两个道具都要能被 undo 撤销。
```

智能体工作：补齐 case03–05，5 个样例全部逐行一致。修复了 bomb 无条件清除失败标志的问题（改为只在真正清空轨道时才清）。将 main.cpp 输入解析改为整份输入按行切分、每条命令独立解析，拒绝 -1、1x、空串等非法计数，兼容 CRLF，解决了原先 std::cin >> 失败后会把下一行命令一起吞掉的错位问题。再做 4000 组随机差分（含非法参数、连续 UNDO），0 处不一致。



5 个样例比对结果

case01 PASS case02 PASS case03 PASS case04 PASS case05 PASS

普通构建（-O2）与 ASan/UBSan 构建下结果相同，零 warning、无泄漏无越界。另外用独立参考实现做了 4000 组随机差分（重点覆盖 UNDO 与道具混合、非法参数），0 处不一致。

本阶段修改

1. main.cpp 输入解析改为逐行处理（修掉一个真实缺陷）

差分测试暴露：原实现用 std::cin >> int 读 FIRE 的参数，失败后 ignore 到行尾，会把**下一行命令**一起吞掉，导致后续命令整体错位（表现为多输出/少输出一个 ERROR）。现在整份输入按行切分，每条命令独立解析：

- parseCount 要求整串都是数字，-1、1x、空串一律拒绝——不再依赖 std::cin 的负数语义。
- 命令 token 数量也校验：FIRE/BOMB/PAINT 恰好 2 个 token，UNDO/SHOW 恰好 1 个，否则 ERROR。
- n=0/q=0 时的空行仍会被读掉，保证后续 q/m 不错位；顺带兼容 CRLF。

2. game.cpp 修正 bomb 对胜负标志的处理

原来无条件 lost_ = false，等于让一次不相干的炸弹悄悄改写了先前 FIRE 得出的失败状态。现在改为**只有炸弹真的清空轨道（won_）时才清除 lost_**，否则保留原标志，胜负互斥关系不破。

胜负判定触发点

只在 Game::fire 与 Game::bomb 的末尾统一判定，init / undo / paint 不判定：

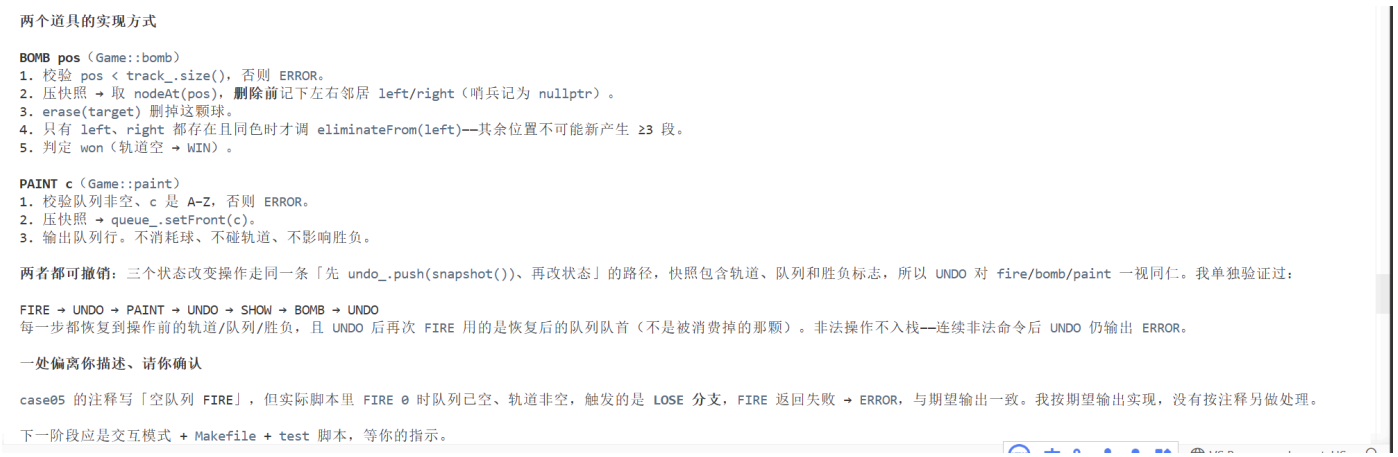
位置	判定	输出
fire	won_ = track_.empty()	轨道行后追加 WIN
fire	lost_ = !won_ && queue_.empty()	轨道行后追加 LOSE
bomb	won_ = track_.empty()	轨道行后追加 WIN；bomb 不消耗队列，永不判负

输出顺序固定为**先轨道行、再 WIN/LOSE**。main.cpp 里 won 优先于 lost 判断，所以两者不会同时出现。

两个道具的实现方式

BOMB pos（Game::bomb）
1. 校验 pos < track_.size()，否则 ERROR。
2. 压快照 → 取 nodeAt(pos)，**删除前**记下左右邻居 left/right（哨兵记为 nullptr）。
3. erase(target) 删掉这颗球。
4. 只有 left、right 都存在且同色时才调 eliminateFrom(left)——其余位置不可能新产生 ≥3 段。
5. 判定 won（轨道空 → WIN）。

PAINT c（Game::paint）



两个道具的实现方式

BOMB pos（Game::bomb）
1. 校验 pos < track_.size()，否则 ERROR。
2. 压快照 → 取 nodeAt(pos)，**删除前**记下左右邻居 left/right（哨兵记为 nullptr）。
3. erase(target) 删掉这颗球。
4. 只有 left、right 都存在且同色时才调 eliminateFrom(left)——其余位置不可能新产生 ≥3 段。
5. 判定 won（轨道空 → WIN）。

PAINT c（Game::paint）
1. 校验队列非空、c 是 A–Z，否则 ERROR。
2. 压快照 → queue_.setFront(c)。
3. 输出队列行。不消耗球、不碰轨道、不影响胜负。

两者都可撤销：三个状态改变操作走同一条「先 undo_.push(snapshot())、再改状态」的路径，快照包含轨道、队列和胜负标志，所以 UNDO 对 fire/bomb/paint 一视同仁。我单独验证过：

FIRE → UNDO → PAINT → UNDO → SHOW → BOMB → UNDO
每一步都恢复到操作前的轨道/队列/胜负，且 UNDO 后再次 FIRE 用的是恢复后的队列队首（不是被消费掉的那颗）。非法操作不入栈——连续非法命令后 UNDO 仍输出 ERROR。

一处偏离你描述、请你确认

case05 的注释写「空队列 FIRE」，但实际脚本里 FIRE 0 时队列已空、轨道非空，触发的是 **LOSE 分支**，FIRE 返回失败 → ERROR，与期望输出一致。我按期望输出实现，没有按注释另做处理。

下一阶段应是交互模式 + Makefile + test 脚本，等你的指示。

阶段 4 · 用户交互

Prompt 思路：补上无参数运行进入的交互模式——开局选预设关卡或自定义、每轮显示带编号轨道与队首/队列/已发射数、命令菜单、消除播报消除了 N 个 X 球、WIN/LOSE 后询问重开。

```
› case05 无误，「空队列 FIRE」就是 FIRE 因队列空返回 ERROR，与期望输出一致。现在，在 MyLab/Linear/Project/src/main.cpp 补上交互模式（无参数运行进入）。

要求
- 无参数运行 → 交互模式；--script → 批处理模式（已实现，别破坏）。
- 开局：提示输入初始轨道（一行颜色）与待发射队列（一行颜色），或用几个预设关卡
  让玩家选（二选一，简单即可）。
- 每轮显示：当前轨道（带位置编号，如 0:R 1:R 2:B …）、队首下一球颜色、
  队列剩余球数、已发射数。
- 命令菜单：fire <pos> / bomb <pos> / paint <c> / undo / show / help / quit。
  非法输入给友好提示，不崩溃。
- 每次 fire/bomb 后，若发生消除，打印一句「消除了 N 个 X 球」（N≥3），
  让连锁消除肉眼可见；这是交互模式额外输出，批处理模式不要有。
- 判定 WIN / LOSE 后打印结果，并询问是否再来一局（重开清空 track/queue/undo）。
- 全部用标准输入输出，中文提示即可（代码文件存 UTF-8）。

验收:交互模式能正常玩通一局：发球→消除→连锁→WIN/LOSE→重开；undo/bomb/paint
  都能用且被 undo 正确回退。把一局的关键截图/文字过程简要汇报。
```

智能体工作：新增 `interactive.h/.cpp`（`main.cpp` 只负责分发交互/批处理）。`Game` 增加 `Elimination`/`lastEliminations()`/`shots()`/`undoSize()`/`colorAt()`/`frontColor()` 等接口，`GameState` 带上 `shots`；`Track` 补 const 版 `nodeAt()`。内置 3 个预设关卡，队列为空时给出只能用 bomb 清球提示。实测一局 `fire 4 → 消除 3 个 B + 4 个 R → WIN → 重开`，自定义局验证 bomb 合并消除、undo 完整恢复、paint、LOSE；10 类非法输入均给针对性提示、无崩溃。

阶段 5 • 收尾（构建 / 测试）

Prompt 思路：收成可一键编译、一键测试的状态——`Makefile`（`EXEEXT` 记账避免 Windows 重编）、`test` 测试脚本（`resolve_exe` 补 `.exe`、逐行比对）、补齐边界样例、README 写足报告。

```
test 测试程序（静态 Python 脚本，文件名就是 test，无扩展名）：
  逻辑：遍历 inputs/case*.txt，逐个用 build/zuma --script 运行（stdin 重定向到样例），
  与 outputs/ 同名文件逐行精确比对，最后打印 PASS/FAIL 汇总，全过则退出码 0。
  必须满足阶段 0 的跨平台要求：
```

智能体工作：新增 `Makefile`、`test`、`.gitignore`，重写 `README.md` 为完整报告；补齐 `case06-10`（单轮消除、非法参数与越界、空轨道空队列、道具组合 + PAINT 撤销、最小 FIRE→WIN）。10 个样例在普通构建与 ASan/UBSan 下全部 PASS、零警告。修复了目标名不带 `$(EXEEXT)` 导致 Windows 每次重编`test` 文件句柄未关闭等问题。

在 MyLab/Linear/Project/ 下新增/重写：

文件	说明
Makefile	EXEEXT / build/zuma$(EXEEXT) / PYTHON ?= python，制表符缩进，每条 recipe 单命令
test	无扩展名 Python 脚本，resolve_exe 补 .exe、stdout utf-8、--script 逐行比对
README.md	完整报告：设计思路、5 个核心算法原理、选型对比、运行说明、复杂度表
.gitignore test	忽略 build/、__pycache__/ 无扩展名 Python 脚本，resolve_exe 补 .exe、stdout utf-8、--script 逐行比对
README.md	完整报告：设计思路、5 个核心算法原理、选型对比、运行说明、复杂度表
.gitignore	忽略 build/、__pycache__/
inputs|outputs/case06-10	补齐样例

SRCS 按实际文件包含 6 个 .cpp（比你给的清单多 interactive.cpp，交互模式需要它）。

验收结果

1. 编译与测试 — PASS（本机用 make，非 mingw32-make）
g++ ... -o build/zuma src/{main,game,track,ball_queue,undo_stack,interactive}.cpp
2. 连跑两次不重编 — PASS
make → "Nothing to be done for 'all'."
用 make OS=Windows_NT 模拟了 Windows 路径：产物为 build/zuma.exe，第二次同样不重编。注意：本机没有 MinGW/Wine，.exe 记账是用 Linux 工具链强制走 Windows 分支验证的，不是真实

.gitignore	忽略 build/、__pycache__/
inputs|outputs/case06-10	补齐样例

SRCS 按实际文件包含 6 个 .cpp（比你给的清单多 interactive.cpp，交互模式需要它）。

验收结果

1. 编译与测试 — PASS（本机用 make，非 mingw32-make）
g++ ... -o build/zuma src/{main,game,track,ball_queue,undo_stack,interactive}.cpp
汇总：10/10 PASS, 0 FAIL
零警告：ASan + UBSan 下 10 个样例同样全部干净通过。另做了负向验证：故意改错一个期望文件，脚本正确报 FAIL 且退出码 1。

三、核心功能函数代码原理解释

3.1 轨道 `Track`：双向链表 + 首尾哨兵

节点结构为 `struct Node { char color; Node* prev; Node* next; }`，首尾各放一个颜色为 `'\0'` 的哨兵节点，链表永远非空。核心操作：

`insertAt(pos, color)`：校验 `0 ≤ pos ≤ size` 后，找到插入位置之后的节点 `next`（`pos == size` 时直接取尾哨兵，追加无需遍历），取 `prev = next->prev`，新建节点接进 `prev` 与 `next` 之间。定位是 `O(min(pos, n-pos))`（`nodeAt` 从较近一端走），链接本身 `O(1)`，尾部追加纯 `O(1)`。

`erase(p)`：校验 `p` 非空且非哨兵后，只做 `p->prev->next = p->next; p->next->prev = p->prev; delete p; --size;`，给定指针时严格 `O(1)`。调用方若之后还要访问 `p` 的邻居，必须在 `erase` 之前先把它们存下来。

3.2 自动消除与连锁消除 `Game::eliminateFrom(Node* p)`

这是整个项目的核心算法。入口是插入/删除后最可能受影响的节点 `p`，循环执行三步：

```
while (p 是真实彩球) {
  1) 以 p 为中心，向左扩张到 first、向右扩张到 last，
     得到包含 p 的极大同色段 [first, last]，长度为 length；
  2) 若 length < 3 —— 不会消除，直接结束；
```

```
  3) 否则：
     a. 先记下删除后将成为左右邻居的 left_ball / right_ball（哨兵记为 nullptr）；
     b. 从 first 删到 last（逐节点 erase）；
     c. 若 left_ball 与 right_ball 都存在且同色，令 p = left_ball 继续下一轮（连锁）；
        否则结束。
}
```

3.3 待发射队列 `BallQueue`：环形缓冲区

底层分配 `cap_ + 1` 个槽，多留一格区分空/满：

```
front_ == back_                  → 空
nextIndex(back_) == front_        → 满
nextIndex(i) = (i + 1 == cap_ + 1) ? 0 : i + 1
```

- `push`：先算 `next = nextIndex(back_)`，若 `next == front_` 返回失败（队列不变），否则写入并回绕 `back_`；
- `pop`：空队抛 `out_of_range`，否则取 `buf_[front_]` 并让 `front_` 回绕；
- `setFront`：`PAINT` 使用，改队首颜色但不移动指针。

3.4 撤销栈 `UndoStack`（快照法）

每次 `FIRE / BOMB / PAINT` 真正改状态之前，把这一刻的完整状态打成 `GameState` 压栈：

```
struct GameState {
    std::vector<char> track;   // 轨道头 → 尾
    std::vector<char> queue;   // 队列队首 → 队尾
    bool won, lost;            // 该时刻的胜负标志
    std::size_t shots;         // 该时刻的累计发射数
};
```

`UNDO` 时弹出栈顶、用它覆盖当前状态，LIFO 语义天然对应撤销上一步。

为什么把胜负标志也放进快照：如果只恢复轨道和队列，FIRE 判负后 UNDO会留下残留的 `lost_` 标志，游戏会被错误判定为已结束。把 `won/lost/shots` 一起快照，撤销后的显示与计数才完全一致。

3.5 用户交互设计

程序支持两种运行模式：

交互模式（无参数运行）：开局可选预设关卡（3 个）或自定义轨道/队列；每轮显示带下标编号的轨道、队首球、队列剩余、已发射数与可撤销步数；支持 `fire / bomb / paint / undo / show / help / quit` 命令，非法输入给出针对性提示；发生连锁消除时逐条播报消除了 N 个 X 球；判定 `WIN / LOSE` 后询问是否再来一局。

批处理模式（`--script`）：从 stdin 读固定格式脚本，输出确定结果，供测试程序自动比对，不输出任何提示语。两种模式由 `main.cpp` 用命令行参数分发，互不影响。

四、运行说明

5.1 编译

在项目目录下：

```
make                # Linux / macOS
mingw32-make        # Windows（MinGW）
```

产物为 `build/zuma`（Windows 下自动为 `build/zuma.exe`）。编译参数 `-std=c++17 -Wall -Wextra -Wpedantic -O2 -g`，零警告。

5.2 测试

```
make test           # 或 mingw32-make test（Windows）
```

`test` 自动遍历 `inputs/case*.txt`，用 `build/zuma --script` 逐个运行，与 `outputs/` 同名文件逐行比对，打印 `PASS/FAIL` 汇总，全部通过退出码 0。

5.3 玩游戏

```
./build/zuma        # Windows: build\zuma.exe
```

5.4 批处理模式（测试用）

```
./build/zuma --script < inputs/case01.txt
```

五、复杂度小结

操作	时间	空间
`Track::insertAt` 定位下标	`O(min(pos, n-pos))`	`O(1)`
`Track::insertAt` 链接（含尾部追加）	`O(1)`	`O(1)`
`Track::erase(节点指针)`	`O(1)`	`O(1)`
`Track::toVector`	`O(n)`	`O(n)`
`BallQueue::push / pop / front / setFront`	`O(1)`	`O(1)`
一次 FIRE / BOMB（含全部连锁消除）	`O(n)`	`O(1)` 额外
一次 PAINT	`O(1)`	`O(1)`
压入快照	`O(n + q)`	`O(n + q)`
一次 UNDO	`O(n + q)`	释放一份快照

其中 `n` 为轨道球数，`q` 为待发射球数。
